SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Sivakumar m	**Reg. No.:**	192521057
Date:	10/10/26	**Duration:**	60 minutes

Code of Conduct

I SIVAKUMAR M (192521057) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:
SIVAKUAMR M

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>*.
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

Agile Project Planning: AI-Generated Code Quality and Security Assurance Platform

1. Identify the Project Vision and Stakeholders

Project Vision

The vision of this project is to develop a platform that automatically checks AI-generated code for quality issues, security vulnerabilities, and errors before it is merged into a project or deployed. It helps developers produce secure, reliable, and maintainable software.

Objectives

Identify errors and security vulnerabilities in AI-generated code.

Check third-party dependencies for known vulnerabilities.

Run automated tests to verify code functionality.

Maintain a record of AI-generated code changes.

Require human approval for high-risk code changes.

Integrate with version control systems and CI/CD pipelines.

Key Stakeholders

Stakeholder	Responsibility
Developers	Submit and improve AI-generated code.
Security Team	Identify vulnerabilities and security risks.
QA/Test Engineers	Perform automated testing and verify results.
Project Manager	Plan tasks, monitor progress, and manage priorities.
Product Owner	Define requirements and prioritize features.
System Administrator	Manage platform configuration and access.
Organization/Client	Ensure software quality and security.

2. Create Epics and User Stories

An Epic is a large project feature that can be divided into smaller user stories. A User Story describes a feature from the user's point of view.

Epic	User Story
Code Quality Analysis	As a developer, I want to scan AI-generated code, so that I can identify coding errors and quality issues.
Security Analysis	As a security engineer, I want to detect insecure coding practices, so that security risks can be reduced.
Dependency Checking	As a developer, I want to scan third-party dependencies, so that vulnerable libraries can be identified.
Automated Testing	As a QA engineer, I want to run automated tests, so that I can verify whether the code works correctly.
CI/CD Integration	As a developer, I want code analysis to run automatically in the CI/CD pipeline, so that unsafe code can be identified before deployment.
AI Code Traceability	As a project manager, I want to track AI-assisted code changes, so that every modification can be traced.
Human Review	As a security reviewer, I want high-risk changes to require approval, so that dangerous code is not merged without review.
Audit and Monitoring	As an administrator, I want to view audit logs and reports, so that I can monitor code quality and security activities.

3. Define Acceptance Criteria

Acceptance criteria specify the conditions that must be satisfied for a user story to be considered complete.

User Story 1: Code Quality Analysis

User Story: As a developer, I want to scan AI-generated code, so that I can identify coding errors and quality issues.

Given the developer has submitted source code,

When the developer starts a code scan,

Then the platform must analyse the code and display detected issues with their severity and location.

User Story 2: Security Analysis

User Story: As a security engineer, I want to detect insecure coding practices, so that security risks can be reduced.

Given source code is available for analysis,

When the security scan is executed,

Then the platform must display detected security issues, their severity, and recommended corrective actions.

User Story 3: Dependency Checking

User Story: As a developer, I want to scan third-party dependencies, so that vulnerable libraries can be identified.

Given the project contains a dependency manifest,

When the dependency scan is performed,

Then the platform must report known vulnerable dependencies and available safer versions when known.

User Story 4: Automated Testing

User Story: As a QA engineer, I want to run automated tests, so that I can verify whether the code works correctly.

Given the project has configured automated tests,

When the test process is started,

Then the platform must display the number of passed and failed tests and provide failure details.

User Story 5: Human Review

User Story: As a security reviewer, I want high-risk changes to require approval, so that dangerous code is not merged without review.

Given a code change is classified as high-risk,

When the developer attempts to merge it,

Then the platform must block the merge until an authorized reviewer approves the change.

4. Prioritize the Product Backlog

The product backlog is the list of features arranged according to their importance, business value, and dependencies.

Priority levels: High, Medium, Low

Priority	User Story / Feature	Business Value
1 – High	Code submission and scanning	Provides the foundation for code analysis.
2 – High	Code quality analysis	Identifies coding errors early.
3 – High	Security vulnerability scanning	Helps prevent insecure code from being merged.
4 – High	Dependency vulnerability checking	Identifies vulnerable third-party libraries.
5 – High	Automated testing	Verifies code functionality.
6 – High	High-risk human approval	Prevents unapproved risky changes.
7 – Medium	CI/CD integration	Automates checks in development workflows.
8 – Medium	AI code traceability	Tracks AI-assisted modifications.
9 – Medium	Audit logs and reports	Supports monitoring and accountability.
10 – Low	Advanced dashboards and analytics	Improves reporting and usability.

The initial focus is on code submission, analysis, and security because these are the foundation of the platform. CI/CD integration and advanced reporting can be expanded in later sprints.

5. Plan the Sprint Backlog

A Sprint Backlog contains the user stories and development tasks selected for a particular sprint.

Sprint 1: Code Analysis Foundation

Selected User Story	Development Tasks
Submit source code	Create a code upload interface; validate file types; store submitted code securely.
Analyse code quality	Integrate a static analysis tool; execute scans; collect issue details.
Display analysis results	Create a results page; show issue descriptions, locations, and severity.
Basic security scanning	Configure security rules; detect selected unsafe coding patterns; display findings.

Sprint 1 Tasks

Design the basic platform interface.

Develop source-code submission functionality.

Integrate a static code analysis tool.

Implement basic security scanning.

Display scan results and issue severity.

Test the code submission and analysis workflow.

6. Estimate Effort Using Story Points

Story points estimate the relative effort, complexity, and uncertainty of completing a user story. The Fibonacci sequence is used: 1, 2, 3, 5, 8, 13.

User Story	Story Points	Justification
Submit source code	3	Requires a simple upload interface and validation.
Code quality analysis	5	Requires integrating and configuring a static analysis tool.
Display analysis results	3	Requires a results page and issue presentation.
Basic security scanning	8	Requires security rules, scan integration, and result validation.
Dependency vulnerability checking	5	Requires dependency file parsing and vulnerability database integration.
Automated testing	8	Requires test execution, result collection, and failure reporting.
High-risk human approval	5	Requires risk classification, reviewer permissions, and approval workflow.
CI/CD integration	8	Requires integration with build pipelines and automated quality gates.

Estimation technique: Planning Poker, in which team members independently estimate each story and discuss differences before agreeing on a final estimate.

For example, basic security scanning receives 8 points because configuring security rules and validating findings is more complex than creating a simple upload interface, which receives 3 points.

7. Present the Sprint Goal and Expected Deliverables

Sprint Goal

To develop a basic working platform that accepts source code, performs code quality and initial security scans, and displays detected issues so that developers can identify problems before merging their code.

Expected Sprint 1 Deliverables

A basic user interface for submitting source code.

Source-code validation and secure handling.

Integration with a static code analysis tool.

Initial security scanning functionality.

A results page showing detected issues and their severity.

Basic functional testing of the scanning workflow.

Expected Outcome

At the end of Sprint 1, developers should be able to submit supported source code, run an analysis, and view code quality and initial security findings. Dependency checking, automated testing, CI/CD integration, audit trails, and human approval workflows can be developed in subsequent sprints.

Conclusion: This Agile plan divides the AI-Generated Code Quality and Security Assurance Platform into manageable epics, user stories, acceptance criteria, prioritized backlog items, and sprint tasks. Using story points and clear sprint goals helps the team develop the platform incrementally while improving software quality, security, traceability, and reliability.